

Track B config (`configs/trackB.yaml`) — what each key changes (train vs eval)

This note maps every key in `local_extraction/configs/trackB.yaml` to:

- **Where it's used** (training / dataset-tokenizer / evaluation / tooling)
- **What it changes** (outputs, metrics, runtime)
- **Why it changes it** (mechanism in the code)
- **How changing it changes results** (expected direction of change)

It's written against the current local runners:

- Training: `local_extraction/trackB/trackB_train_loader.py`
 - Dataset + schema parsing + binning: `local_extraction/trackB/trackB_dataset.py`
 - Tokenizer/backbone: `local_extraction/trackB/trackB_tokenizer.py`
 - Evaluation: `local_extraction/trackB/trackB_eval.py`
-

How config is loaded (important nuance)

- Training supports presets via `--config` :
 - `python -m trackB.trackB_train_loader --config trackB` (default)
 - `python -m trackB.trackB_train_loader --config trackB_resnet18_baseline`
 - `python -m trackB.trackB_train_loader --config trackB_videomae_ego`
 - Evaluation currently loads **only** `trackB` (hard-coded):
 - `local_extraction/trackB/trackB_eval.py` calls `load_config('trackB')` .
 - So changing `trackB_videomae_ego.yaml` does **not** affect eval unless you also update `trackB.yaml` or change eval to load a preset.
 - Evaluation does, however, read key settings from the **checkpoint** (`train_config` embedded in the `.pt`) to stay consistent with training (notably `tokens_root` and `video_backbone`).
-

Global / shared keys

version

- **Used by:** mostly config interpolation (e.g., hotspot priors path)
- **Effect:** chooses the dataset version folder under `local_extraction/` when a path uses `${version}` .
- **Why:** `base.yaml` defines paths like `${paths.local_extraction}/${version}/...` .
- **Changing it changes results by:** pointing to different data assets (frames/labels/priors) *if* the consuming script uses the interpolated path.

`paths.*` (from `base.yaml`) vs Track B scripts

Track B scripts are **not fully consistent** about which `paths.*` keys they read:

- `base.yaml` defines keys like `paths.frames_root` , `paths.manifests_root` , `paths.runs_root` .
- `trackB_eval.py` looks for `paths.extracted_frames` , `paths.manifests` , and `paths.runs` .
- `trackB_train_loader.py` currently **ignores YAML paths** for frames and uses a hard-coded `local_extraction/v2/extracted_frames` .

Practical takeaway:

- For training, treat frames root as fixed to `local_extraction/v2/extracted_frames` unless you edit code.
- For evaluation, if you want strict reproducibility / non-default roots, consider adding these keys under `paths:` in `trackB.yaml` :
 - `paths.extracted_frames`
 - `paths.manifests`
 - `paths.runs`

Model keys (`model.*`) — architecture & capacity

`model.tokenizer.video_backbone`

- **Used by:** training + tokenizer + evaluation
- **Effect:** selects the feature extractor:
 - `"resnet18"` : per-frame 2D grid tokens, 512-dim before projection
 - `"videomae_ego"` : VideoMAE wrapper, 768-dim before projection
- **Why:** `TokenizerConfig.video_backbone` is read from YAML, and training stores it into the checkpoint `train_config` .
- **Changing it changes results by:** changing the representation quality/domain match (often largest single quality lever), and changing runtime/memory.

`model.tokenizer.img_size`

- **Used by:** tokenizer
- **Effect:** resize for backbone input.
- **Why:** tokenizer transform uses `Resize((img_size, img_size))` .
- **Changing it changes results by:** trading detail vs speed; changing it also changes token grid resolution indirectly.

`model.tokenizer.time_len` , `model.tokenizer.time_stride`

- **Used by:** tokenizer + dataset
- **Effect:** defines the temporal window used to build video tokens.
- **Why:** dataset samples a window of length `time_len` ending at the current frame, stepping back by `time_stride` .
- **Changing it changes results by:**
 - larger `time_len` (more context): potentially better action disambiguation but slower

- larger `time_stride` (wider temporal span): captures longer-term cues but can miss short actions

`model.tokenizer.videomae.*`

- **Used by:** tokenizer when `video_backbone="videomae_ego"`
- **Effect:** controls VideoMAE wrapper construction and weight loading.
- **Why:** wrapper reads: `weights_path`, `patch_size`, `tubelet_size`, `embed_dim`, `depth`, `num_heads`, `freeze_encoder`.
- **Changing it changes results by:**
 - wrong `weights_path`: falls back to random init → results degrade
 - `freeze_encoder=false`: allows finetuning (slower, potentially higher ceiling)

Note: Some keys present in the preset `trackB_videomae_ego.yaml` (like `hf_model_name`, `num_frames`) are not currently consumed by the wrapper code.

`model.projector.in_dim`

- **Used by:** training (as default)
- **Effect:** input feature dimension expected from backbone tokens.
- **Why:** projector is `Linear(in_dim, out_dim)`.
- **Changing it changes results by:**
 - if mismatched to real token dim, you'll get a shape error (hard failure)
 - otherwise, no "quality knob" effect (it's a compatibility setting)

Important nuance: if `data.tokens_root` is set and tokens exist, training attempts to **auto-detect** token dimension from the first token file and overrides `in_dim`.

`model.projector.out_dim`

- **Used by:** training + evaluation
- **Effect:** sets the fused token embedding dimension `C` used by fusion + head.
- **Why:** `TOKEN_DIM = model.projector.out_dim`.
- **Changing it changes results by:** capacity/speed tradeoff (bigger `C` → more capacity + slower).

`model.fusion.layers`

- **Used by:** training + evaluation
- **Effect:** number of fusion blocks.
- **Why:** `TrackBFusion(FusionConfig(..., layers=...))`.
- **Changing it changes results by:** capacity/speed tradeoff.

Note: `model.fusion.heads` and `model.fusion.dropout` exist in YAML but are not currently passed into `FusionConfig` in the training/eval scripts.

`model.head.hidden`

- **Used by:** training
- **Effect:** MLP hidden width inside the prediction head.
- **Why:** training passes `hidden=HEAD_HIDDEN` into `HeadConfig`.

- **Changing it changes results by:** capacity/speed tradeoff.

Note: `model.head.num_classes` is not used to set output size during training; training infers the number of classes from manifest labels (minimum 2).

Multi-task keys (`multi_task.*`) — what is learned

`multi_task.enabled`

- **Used by:** training
- **Effect:** enables noun/verb auxiliary heads *if labels exist in manifests*.
- **Why:** training uses it as `use_multi_task_labels` and builds noun/verb vocab from observed `noun_id` / `verb_id`.
- **Changing it changes results by:**
 - `true` : can improve representation and next-active via auxiliary supervision (if labels are present)
 - `false` : trains only next-active + TTC regression/bin loss

`multi_task.ttc_mode`

- **Used by:** training + evaluation (partially)
- **Effect:**
 - `"reg"` : TTC is trained as a scalar regression
 - `"bin"` : TTC bin classification head is enabled (if bins exist)
- **Why:** training uses `ttc_mode == 'bin'` to enable bin loss.
- **Changing it changes results by:**
 - regression: finer-grained TTC, evaluated via MAE
 - bins: coarser TTC but can be easier to learn/compare

`multi_task.ttc_thresholds`

- **Used by:** dataset + evaluation
- **Effect:** defines the bin edges (in seconds) used for TTC binning.
- **Why:** dataset uses these thresholds in `ttc_to_bin()`.
- **Changing it changes results by:** changing the class boundaries for TTC bins → changes bin labels and any bin-based metrics.

`multi_task.loss_weights.*`

- **Used by:** training
- **Effect:** weights of each loss term in the total loss.
- **Why:** training sums: `next + ttc + noun + verb` with these weights.
- **Changing it changes results by:** shifting optimization focus (e.g., higher noun/verb can help semantics but may hurt next-active if over-weighted).

Notes:

- Defaults in code differ from YAML comments (e.g., `next_active` default 1.5 in code), but since YAML sets explicit values, YAML wins.
- In the *demo standalone* loop, TTC loss weight is additionally scaled by `0.1` (demo-only behavior).

`multi_task.predict_noun / predict_verb / predict_ttc`

- **Used by:** currently not used by the training code paths.
- **Effect:** none right now.
- **Why:** training gates multi-task behavior via `multi_task.enabled` and label presence.

`multi_task.ttc_bins`

- **Used by:** not used.
 - **Effect:** none.
 - **Why:** the number of TTC bins is inferred from data / `ttc_to_bin` (typically 4 bins with 3 thresholds).
-

Training keys (`training.*`) — optimization & reproducibility

`training.epochs , training.batch_size`

- **Used by:** training
- **Effect:** training length and batch size.
- **Changing it changes results by:** longer training can improve metrics; larger batch may stabilize gradients but needs memory.

CLI overrides exist:

- `--epochs , --batch_size` .

`training.lr , training.min_lr , training.warmup_epochs`

- **Used by:** training
- **Effect:** learning-rate schedule (warmup then cosine decay).
- **Changing it changes results by:** convergence stability and final quality.

CLI override exists:

- `--lr` .

`training.label_smoothing`

- **Used by:** training
- **Effect:** cross-entropy label smoothing.
- **Changing it changes results by:** can improve calibration/robustness; too high can reduce max accuracy.

`training.weight_decay`

- **Used by:** not used.
- **Effect:** none currently.
- **Why:** optimizer is `Adam(...)` without `weight_decay` .

`training.candidate_limit`

- **Used by:** training + evaluation + dataset
- **Effect:** caps number of candidate boxes per sample/frame.
- **Why:** ROI pooling and heads scale with number of boxes.
- **Changing it changes results by:**
 - higher → better chance the true positive box is included, but slower
 - lower → faster, but may miss the relevant object

`training.normalize_ttc`

- **Used by:** training + evaluation + dataset
- **Effect:** z-score normalizes TTC targets/predictions using dataset stats.
- **Changing it changes results by:** can stabilize TTC training; if stats are poor (too few samples), it can add noise.

`training.save_epoch_checkpoints` , `training.save_best_checkpoint`

- **Used by:** training
- **Effect:** writes checkpoints each epoch and/or best checkpoint.
- **Changing it changes results by:** doesn't change model quality; changes what artifacts exist.

`training.eval_every`

- **Used by:** training
- **Effect:** validation frequency (epochs).

`training.early_stopping.*`

- **Used by:** training (partially)
- **Effect:** early stopping uses `patience` , `monitor` , `mode` .
- **Changing it changes results by:** can stop training earlier and prevent overfitting.

Note: `training.early_stopping.enabled` exists in YAML but is not currently checked; patience-based stopping always runs.

`training.amp`

- **Used by:** training
- **Effect:** enables CUDA AMP.
- **Changing it changes results by:** usually same metrics, faster on GPUs (some numerical differences possible).

Demo keys (`demo.*`) — quick sanity checks

`demo.enabled`

- **Used by:** training
- **Effect:** if true, training switches to demo mode.
- **Why:** training sets mode based on `demo.enabled` .

Note: `training.mode` exists in YAML but is not used.

`demo.steps`, `demo.variant`, `demo.use_real_labels`

- **Used by:** training
- **Effect:** how the demo loop runs and whether it samples from real manifests.

Evaluation keys (`evaluation.*`) — post-training scoring/overlays

`evaluation.batch_size`

- **Used by:** eval
- **Effect:** evaluation throughput.

`evaluation.save_overlays`, `evaluation.topk_overlay`

- **Used by:** eval
- **Effect:** whether to write qualitative overlay images and how many top candidates.

`evaluation.hotspot_priors.enabled`, `.path`, `.alpha`

- **Used by:** eval
- **Effect:** blends next-active score with a prior score based on predicted (noun, verb).
- **Why:** eval computes: $s \leftarrow (1-\alpha) s + \alpha \cdot \text{prior}(\hat{n}, \hat{v})$.
- **Changing it changes results by:**
 - higher `alpha` : relies more on priors; can help if noun/verb are accurate and priors are good
 - wrong/old priors file: can hurt ranking

`evaluation.clip_rerank.enabled`, `.model`, `.weight`

- **Used by:** eval
- **Effect:** further blends candidate score with CLIP similarity between crop and predicted noun prompt.
- **Why:** eval computes: $s \leftarrow (1-w) s + w \cdot \text{CLIP}(\text{crop}, \hat{n})$.
- **Changing it changes results by:**
 - higher `weight` : stronger CLIP influence; can improve ranking but is slower

`evaluation.compute_map`, `evaluation.compute_ttc_mae`

- **Used by:** not used.
- **Effect:** none currently.

Data keys (`data.*`) — which manifests/tokens are used

`data.stageB_run`

- **Used by:** training + evaluation
- **Effect:** selects which Track A Stage B run provides `head_train` / `head_val`.
- **Why:** if null, scripts auto-detect latest Stage B run under `local_extraction/runs/Track_A`.

- **Changing it changes results by:** changing the training/eval dataset (different candidate generation and cropping policy upstream).

`data.train_manifest` , `data.val_manifest`

- **Used by:** training (both) and evaluation (val only)
- **Effect:** overrides which manifest files to use.
- **Why:** if null, scripts try to resolve `head_train` / `head_val` from StageB run.
- **Changing it changes results by:** changing samples/labels/candidates used to train/evaluate.

`data.tokens_root`

- **Used by:** training; evaluation uses the value stored in checkpoint `train_config` .
- **Effect:** uses pre-extracted token files instead of running the backbone on-the-fly.
- **Why:** dataset can load tokens from `.pt` files for each frame.
- **Changing it changes results by:** usually does not change metrics (if tokens match), but massively changes runtime.

Smoke test (`smoke_test.*`)

- **Used by:** not used by the main training/eval scripts directly.
- **Effect:** acts as documentation or hooks for separate tooling.

Summary (what to change for different outcomes)

High-impact knobs (usually change metrics, not just runtime):

- `model.tokenizer.video_backbone` : ResNet18 vs VideoMAE (domain match)
- `model.tokenizer.time_len/time_stride` : temporal context
- `data.stageB_run` / `data.*_manifest` : which StageB-produced dataset you learn from
- `training.candidate_limit` : recall-vs-speed tradeoff (missing the true positive kills performance)
- `multi_task.enabled` and `multi_task.loss_weights.*` : how much you rely on noun/verb/TTC supervision

High-impact runtime knobs (often keep metrics similar):

- `data.tokens_root` : pre-extracted tokens vs on-the-fly backbone
- `training.amp` : faster GPU training
- `training.batch_size` : throughput vs memory

Keys present but currently not wired (no effect unless code is updated):

- `training.weight_decay` , `training.mode`
- `evaluation.compute_map` , `evaluation.compute_ttc_mae`
- `multi_task.predict_*` , `multi_task.ttc_bins`
- some `model.tokenizer.*` flags (`pretrained` , `freeze_backbone`) and `model.fusion.heads/dropout`

Numerical examples (illustrative comparisons)

These are simplified examples meant to show *directional* effects. Real numbers depend on your data, StageB run, and checkpoint.

Example 1 — Candidate limit vs compute

Assume per frame there are on average 24 candidates, but only 1 is correct.

<code>training.candidate_limit</code>	Avg candidates kept	Expected effect on recall	Expected effect on speed
8	8	lower chance GT survives	fast
16	16	better GT retention	medium
32	24	best GT retention (caps not binding)	slow

Example 2 — ResNet18 vs VideoMAE (capacity/domain match)

Backbone (<code>model.tokenizer.video_backbone</code>)	Typical token dim	Typical <code>projector.in_dim</code>	Expected effect
<code>resnet18</code>	512	512	strong baseline, fast, good if tokens_root exists
<code>videomae_ego</code>	768	768	often better egocentric semantics, slower, needs correct weights/tokens

Example 3 — Temporal window choice

<code>time_len</code>	<code>time_stride</code>	Window span (approx)	Expected behavior
8	2	~16 frames back	good short-term cues, moderate compute
16	2	~32 frames back	more context, higher compute
16	4	~64 frames back	longer-term context, may miss quick transitions

Example 4 — Hotspot prior blending (eval ranking)

Assume base next-active score $s=0.60$ and hotspot prior score $p=0.90$.

<code>evaluation.hotspot_priors.alpha</code>	Final score $s'=(1-\alpha)s+\alpha p$
0.0	0.60

<code>evaluation.hotspot_priors.alpha</code>	Final score $s' = (1 - \alpha)s + \alpha p$
0.3	0.69
0.7	0.81

Example 5 — Tokens root (runtime)

<code>data.tokens_root</code>	Training behavior	Expected speed
<code>null</code>	run backbone on-the-fly in dataset	slowest
<code>v2/resnet18_tokens</code>	load cached ResNet tokens	much faster
VideoMAE tokens path	load cached VideoMAE tokens	faster than on-the-fly VideoMAE